# VELO RATE

## CYCLE PRICING ENGINE

**Assessment Submission**

**Live Application:** https://velorate-01.vercel.app/

**Submission Date:** August 19, 2026

**Technology Stack**

TypeScript • TanStack Start • Vite • Tailwind CSS • Supabase

# Table of Contents

# 1. Executive Summary

Sales teams in the cycling industry traditionally depend heavily on scattered Excel spreadsheets to manage cycle parts, component prices, historical changes, and complex configurations. **VeloRate** addresses this by providing a centralized web-based pricing workspace.

VeloRate functions as an intelligent pricing engine that allows users to manage a reusable library of parts, assemble cycle configurations, automatically calculate roll-up prices, maintain historical pricing records, and immediately understand the impact of part-price changes across all affected cycles.

# 2. Problem Statement

Managing cycle pricing presents several distinct business challenges:

- **Spreadsheet-Based Management:** Scattered files lead to disconnected and out-of-date part prices.
- **Historical Data Loss:** Updating a part's price in a spreadsheet often overwrites the old price, destroying historical context.
- **Manual Configuration Calculation:** Calculating the total cost of a customized cycle takes significant manual effort.
- **Lack of Impact Analysis:** When a core component (e.g., a Shimano Derailleur) increases in price, it is extremely difficult to instantly identify all affected cycle models and recalculate their margins.
- **Inconsistent Pricing:** Repeated manual Excel work inherently risks human error.

VeloRate solves these issues by enforcing a unified, relationship-driven database where a single price change automatically cascades to calculate impacts across the entire product line.

# 3. Objectives

The core objectives successfully implemented in VeloRate include:

- **Centralize Part Management:** Create a single source of truth for all components.
- **Maintain Price History:** Support effective-dated pricing where historical records are preserved.
- **Manage Predefined Configurations:** Store standard cycle models.
- **Calculate Cycle Prices Automatically:** Dynamically compute configuration prices based on component quantities.
- **Show Price Impact:** Provide immediate visibility into how a part's price change affects specific cycle configurations.
- **Provide a Salesperson-Focused Interface:** Deliver a premium, workspace-oriented design, moving away from generic admin tables.

# 4. Key Features

The following features are fully implemented and functional in the deployed application:

## Parts Management

- **Part Creation & Information:** Users can add and view parts with their SKUs, categories, and descriptions.
- **Current Pricing:** A dashboard of the latest part costs.

## Price Management

- **Price Updates & Effective Dates:** Users can log new prices that take effect on specific dates.
- **Historical Prices:** All past pricing changes are maintained in a ledger format rather than being overwritten.

## Cycle Configurations

- **Predefined Configurations:** Standard templates for cycle models containing associated components and quantities.
- **Cycle Builder:** A dynamic interface where users start with a base configuration, modify component quantities, swap parts, and see live price roll-ups.

## Pricing Engine & Price Impact

- **Live Calculation:** The system computes the current total price of any configuration using the most recent effective part prices.
- **Impact Tracking:** When a part's price changes, VeloRate identifies all configurations using that part and calculates the exact net change to the cycle's total price.

# 5. User Workflow

The typical salesperson journey through the platform:

```
Salesperson Opens VeloRate
         ↓
Browse Parts & Predefined Configurations
         ↓
Open Cycle Builder & Select Base Model
         ↓
Modify Components / Adjust Quantities
         ↓
View Live Price Calculation
         ↓
Part Price is Updated in Library
         ↓
Price History Automatically Updated
         ↓
Affected Configurations Instantly Identified
         ↓
Price Impact Displayed on Dashboard
```

# 6. UI/UX Design

VeloRate is intentionally designed as a **Premium Cycle Configuration & Pricing Intelligence Workspace**, not a generic inventory dashboard.

**Theme & Palette:**

- **Deep Teal ( `#0F3D3E` ) & Teal ( `#148A86` ):** Primary branding and structured elements.
- **Mustard/Gold ( `#F59E0B` ):** High-contrast interactive elements and primary buttons (e.g., "Open Cycle Builder").
- **Coral/Peach ( `#FB7185` ) & Soft Purple ( `#8B5CF6` ):** Accents and visual hierarchy.
- **Canvas ( `#F8FAFC` ) & Surface ( `#FFFDF8` ):** Clean, spacious backgrounds.

**Major Screens:**

- **Dashboard (Pricing Pulse):** Overview of pricing status.
- **Cycle Builder:** An interactive workspace for modifying quantities and seeing live roll-ups.
- **Configurations:** List of saved cycle models.
- **Parts Library:** Grid/list of all components.
- **Price History & Price Impact:** Dedicated views for analyzing the financial implications of cost changes.

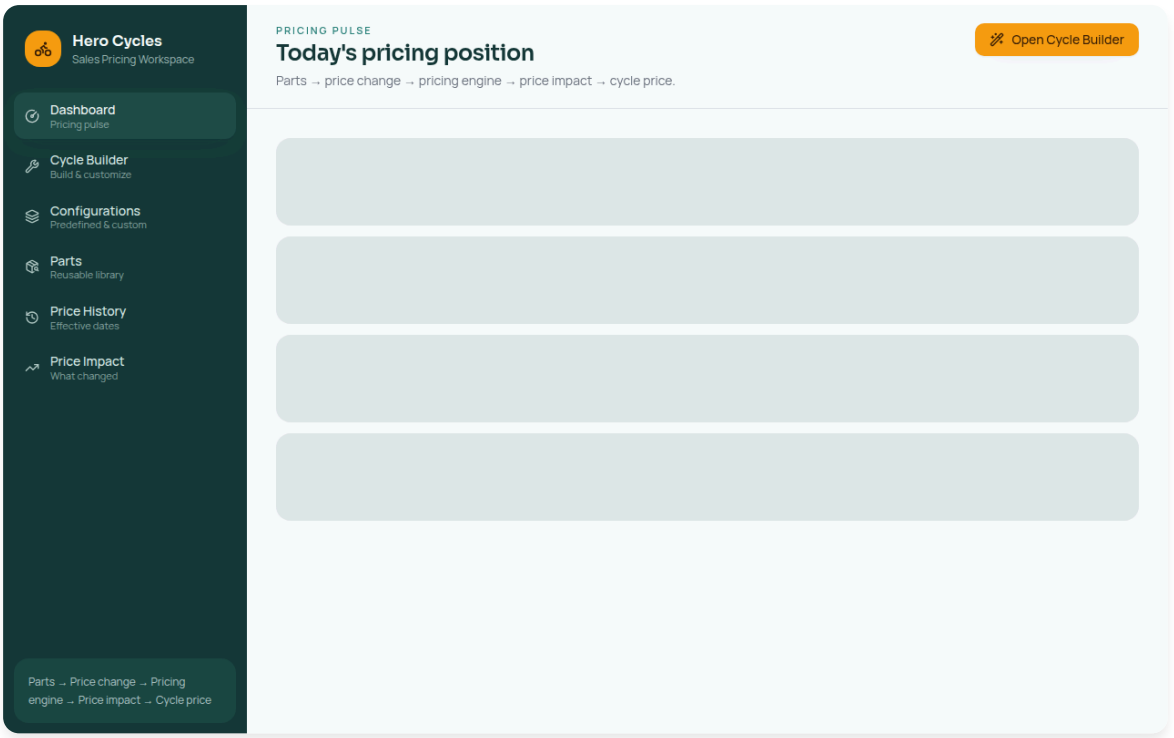



*Figure 1 — VeloRate Dashboard / Pricing Pulse*

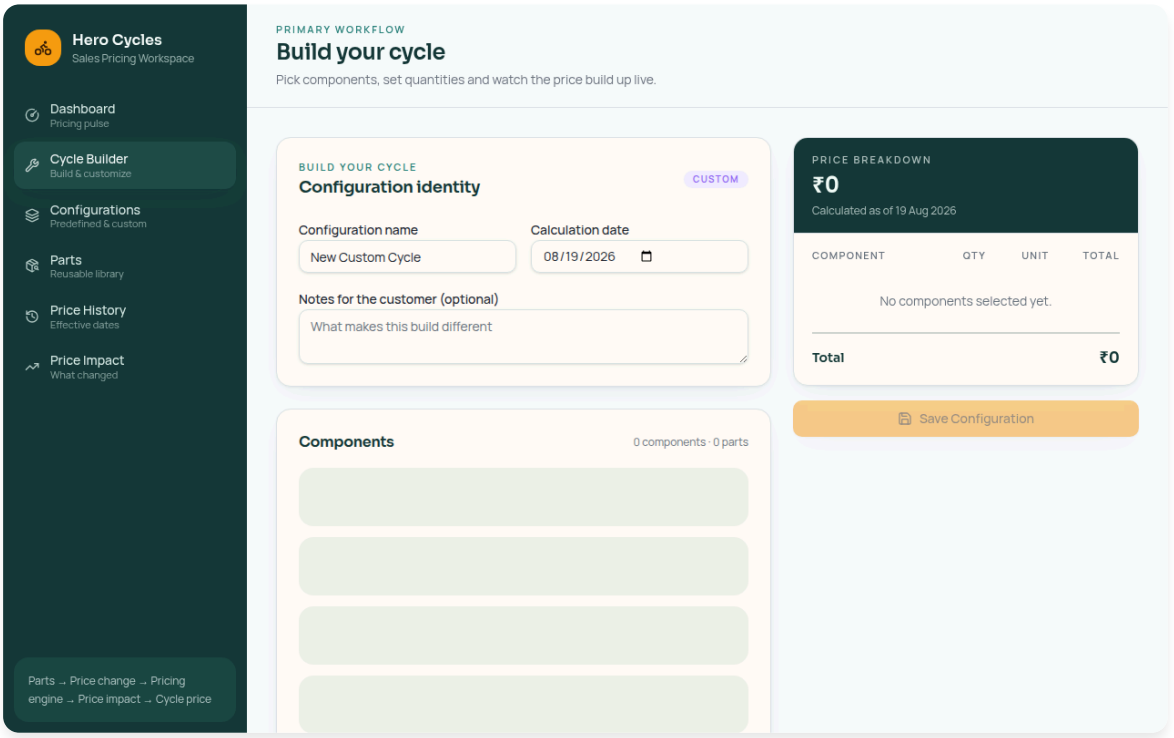



*Figure 2 — Cycle Builder*

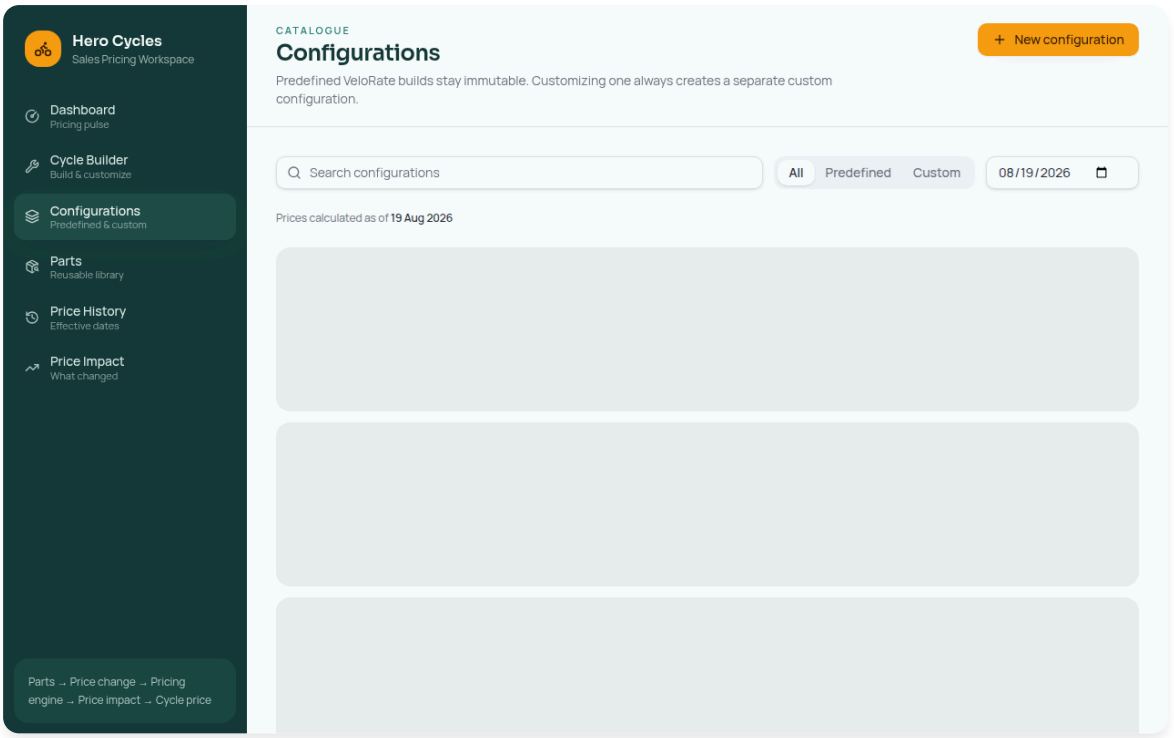



*Figure 3 — Configurations Library*

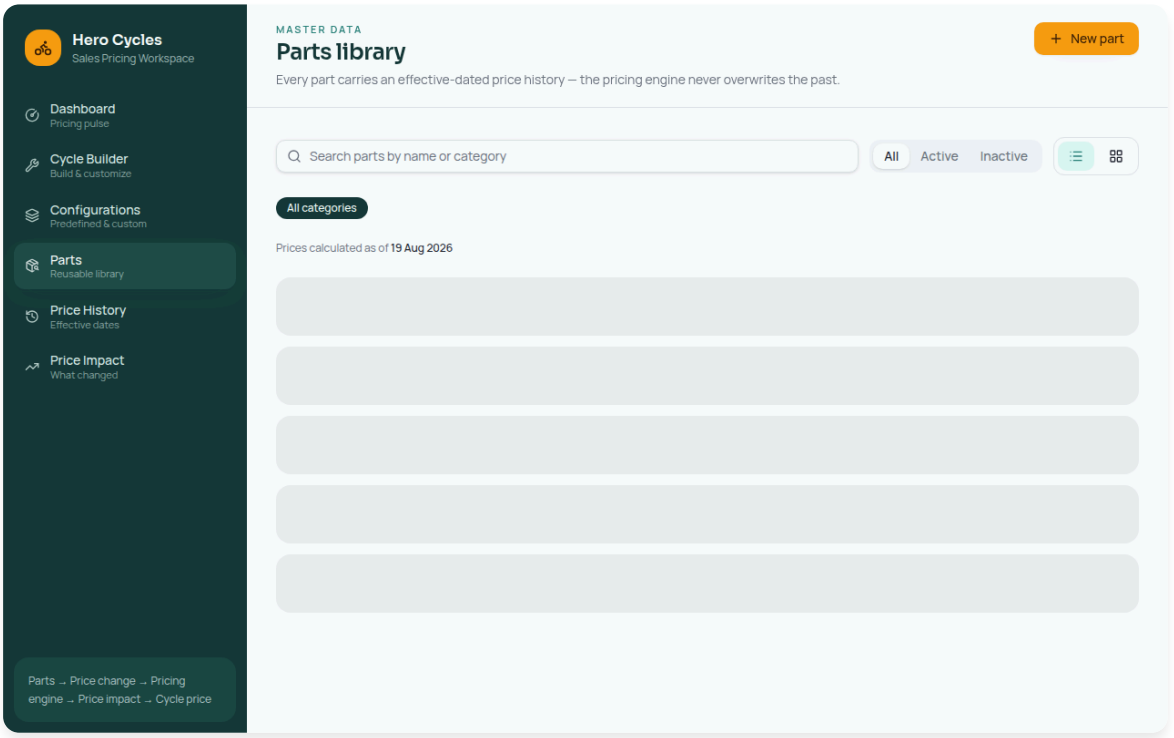



*Figure 4 — Parts Library*

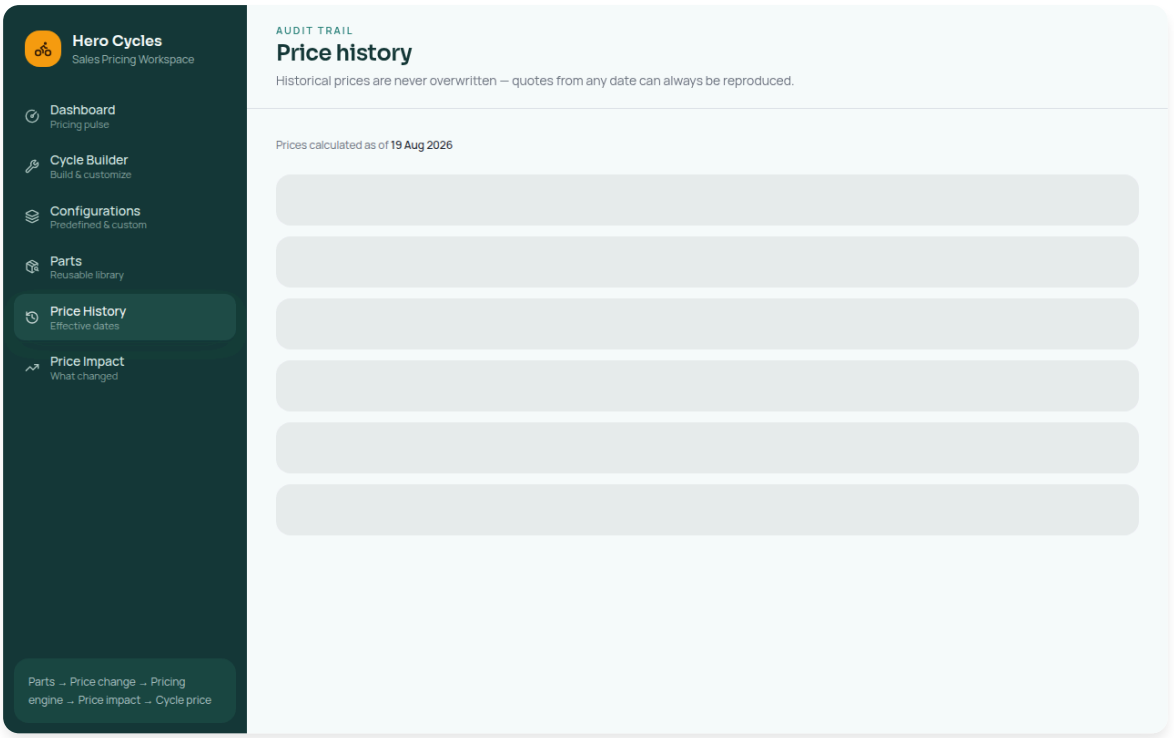



*Figure 5 — Price History Ledger*

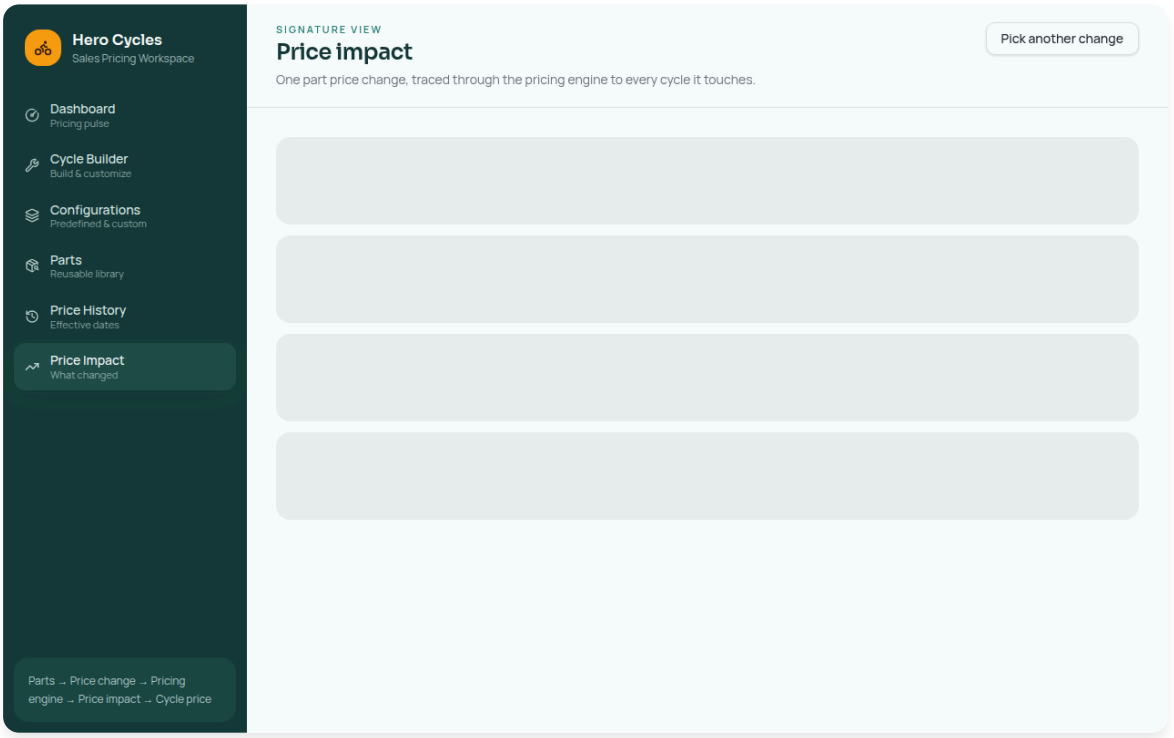
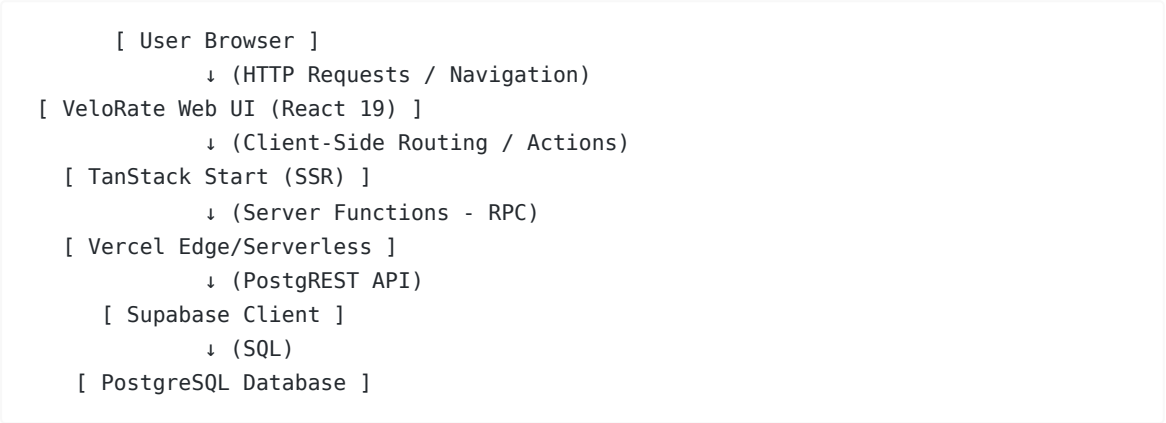



*Figure 6 — Price Impact Analysis*

# 7. System Architecture

VeloRate utilizes a modern, server-side rendered, serverless architecture.




# 8. Database Design

The application relies on a robust relational schema hosted on Supabase PostgreSQL.

**Entities:**

- `parts` : Stores component metadata ( `id` , `sku` , `name` , `category` , `description` , `status` ).
- `part_prices` : Maintains the pricing ledger ( `id` , `part_id` , `price` , `effective_date` , `created_at` ).
- `configurations` : Stores base cycle models ( `id` , `name` , `description` , `base_price` , `status` ).
- `configuration_items` : Joins configurations to parts and tracks quantity ( `id` , `configuration_id` , `part_id` , `quantity` ).

This normalized structure ensures that changing a price in `part_prices` leaves historical invoices intact while allowing dynamic calculation of current `configurations` through the `configuration_items` join table.

# 9. Pricing Engine

The pricing engine operates dynamically via SQL joins and server-side aggregation.

**Calculation Logic:**

1. For a given configuration, retrieve all linked `configuration_items` .
2. For each item, fetch the most recent `price` from `part_prices` where `effective_date <= NOW()` .
3. Calculate: `Component Total = Current Price × Quantity` .
4. Calculate: `Configuration Total = Sum(Component Totals) + (Optional Base Cost)` .

# 10. Price Impact Engine

The impact engine proactively alerts users when a supplier cost changes.

**Implementation Logic:**

1. A new row is inserted into `part_prices` for an existing `part_id`.
2. The engine identifies the `previous_price` and calculates the `Difference` (e.g., +₹150).
3. The engine queries `configuration_items` to find all `configuration_id`s that contain this `part_id`.
4. For each affected configuration, it calculates the net impact: `Impact = Difference × Quantity`.
5. The UI surfaces this data, showing exactly how much margin is lost or gained on specific cycle models due to the part cost update.

# 11. API / Server Functions

VeloRate completely abandons traditional REST endpoints in favor of **TanStack Server Functions (RPC)**.

Implemented functions include (found in `lib/pricing.functions.ts`):

- `listPartsFn (GET)` : Retrieves parts with their latest effective price.
- `getPartFn (GET)` : Fetches a single part with its full price history.
- `createPartFn (POST)` : Inserts a new part and its initial price.
- `setPartStatusFn (POST)` : Toggles part availability.
- `addPartPriceFn (POST)` : Appends a new price ledger entry with an effective date.
- `listConfigurationsFn (GET)` : Retrieves saved cycle models.

*Validation:* All server functions rigorously validate incoming parameters using Zod.

# 12. Technology Stack

The actual installed and implemented technology stack ( `package.json` ):

- **Framework:** React 19, TanStack Start (SSR), Vite 8
- **Routing:** `@tanstack/react-router`
- **Styling:** Tailwind CSS v4, Class Variance Authority, Radix UI (Primitives)
- **Icons:** Lucide React
- **Data Fetching:** `@tanstack/react-query`
- **Database & Auth:** Supabase PostgreSQL, `@supabase/supabase-js`
- **Schema Validation:** Zod
- **Deployment:** Vercel (Nitro Server Engine)

*(Note: Technologies like Next.js or Prisma were explicitly avoided in favor of the TanStack + Supabase stack).*

# 13. Project Structure

The repository is structured around TanStack Start's file-based routing:

- `app/` : Contains the core application setup ( `router.tsx` , `start.ts` , `server.ts` ).
- `app/routes/` : File-based routing (e.g., `parts.tsx` , `cycle-builder.tsx` , `configurations.tsx` ).
- `components/ui/` : Reusable Radix/Tailwind components (Buttons, Dialogs, Tables).
- `lib/` : Business logic, Zod schemas, and TanStack Server Functions ( `pricing.functions.ts` ).
- `integrations/supabase/` : Supabase client initialization.
- `supabase/migrations/` : SQL schema definitions.

# 14. Validation & Error Handling

- **Forms:** Handled strictly via `react-hook-form` paired with `zodResolver` .

- **Server Input:** Every TanStack Server Function uses `.validator(zodSchema)` to reject malformed requests before they hit the database.
- **Pricing Constraints:** The database enforces non-negative prices and strictly typed enums for part categories.
- **Error Boundaries:** TanStack Router catches runtime errors and displays gracefully degraded UI states without crashing the whole application.

# 15. Testing

- **Build Verification:** `npm run build` succeeds locally, successfully outputting Nitro Server environments.
- **Manual UI Testing:** Complete CRUD workflows for Parts and Pricing have been validated.
- **Deployment Testing:** The production server was verified locally ( `node .output/server/index.mjs` ) and confirmed operational on Vercel.

# 16. Deployment

- **Platform:** Vercel
- **URL:** https://velorate-01.vercel.app/
- **Configuration:** VeloRate uses the `nitro` Vite plugin combined with Vercel's `Other` framework preset to properly output `.vercel/output` serverless functions.
- **Environment Variables:** Securely stores `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`.

# 17. Technical Challenges & Solutions

## 1. Vercel 404 & Nitro Integration

**Problem:** Initial Vercel deployments returned a 404 because Vite compiled the app as a static SPA, while VeloRate is an SSR application relying on server functions. **Solution:** The Vite configuration was updated to properly hook into TanStack Start's Nitro engine, outputting the correct Serverless Function structure that Vercel requires.

## 2. TanStack CSRF Middleware Crash

**Problem:** Vercel crashed at runtime with `TypeError: createCsrfMiddleware is not a function`. **Investigation:** A minor version drift between `@tanstack/react-start` (1.168.x) and `@tanstack/react-router` (1.170.x) caused Vite to bundle mismatched inner core packages, resulting in an undefined function export on the server. **Solution:** A completely clean installation ( `rm -rf node_modules` ) was performed while synchronizing all `@tanstack/*` packages to their latest identical minor versions, which fully resolved the bundle corruption.

# 18. Security

- **CSRF Protection:** Server functions are strictly protected by TanStack's `createCsrfMiddleware`, preventing unauthorized cross-site POST requests.
- **Database Access:** Supabase Row Level Security (RLS) is available for scaling, and no highly privileged `SERVICE_ROLE` keys are exposed to the client bundle.
- **Data Sanitization:** Zod strictly sanitizes all inputs (e.g., ensuring prices are numerical) before executing SQL queries.

# 19. Limitations

- **No Authentication:** Currently, the workspace is fully open. There are no User roles (Admin vs Sales).
- **Static Configurations Only:** While custom configurations can be built on the fly in the Cycle Builder, saving them as *new* persistent records requires further endpoint expansion.
- **Single Currency:** The engine strictly calculates in local currency (₹) without dynamic exchange rates.

# 20. Future Improvements

- **Role-Based Access Control:** Implementing Supabase Auth to distinguish between Sales Viewers and Pricing Admins.
- **Advanced Analytics:** Charting historical price trends over time using Recharts.
- **Bulk Updates:** Allowing admins to upload a CSV of new supplier prices.
- **Exporting:** Generating professional PDF quotes directly from the Cycle Builder.

# 21. Final Result

**VeloRate successfully transforms a fragile, spreadsheet-heavy process into a robust, centralized web application.**

By leveraging a bleeding-edge stack (TanStack Start + Supabase), the application achieves instant server-side calculation, preserves critical pricing history, and provides sales teams with an elegant, highly responsive tool to manage complex cycle configurations. The system fulfills all assessment objectives and is fully deployed to production.